CM3020 Artificial Intelligence

Midterm Coursework

Coursework: Part B Report

Report Title: Genetic Algorithm Optimisation for Mountain Climbing in PyBullet: Population Size and Encoding Scheme Evaluation

Student Name: Ian Alexander Park
Student ID (SID): 230656596

Word Count (excluding references and cover page): ~ 1900

Submission Date: 7 January 2026

1. Introduction

This project adapts the Genetic Algorithm creatures codebase to a new PyBullet environment where the creature must climb a central mountain inside a sandbox. The aim is to evolve creatures that can reach as high as possible while avoiding behaviour that resembles flying.

To meet the coursework requirements, I integrated the GA evaluation loop into the mountain environment, replaced the original fitness function with a mountain climbing objective then carried out controlled experiments that vary GA settings such as population size.

I performed additional experiments that modify the decoding and encoding scheme, including motor control design, part shapes and which parts of the body are evolvable, and summarised results using graphs and tables.

The report presents two groups of experiments. First, basic GA setting sweeps compare population sizes of 10, 25 and 50 using the same encoding scheme. Second, encoding scheme experiments compare multiple genotype to phenotype mappings under a fixed population size of 25. Across both groups, performance is assessed primarily by how close the creature gets to the peak while being supported by the terrain, with airtime used as an anti cheating metric.

2. System overview

2.1 Environment and task

The environment is a sandbox containing a mountain mesh. Each evaluation starts by spawning a procedurally generated creature above the terrain and simulating it for a fixed number of physics steps. During simulation, the creature's motors produce joint torques based on its genome encoded control parameters. The task objective is to move up the mountain and approach the peak without relying on prolonged airtime.

2.2 Genetic algorithm loop

Each genome defines a creature through a decoding process that maps gene values to morphology parameters such as link lengths, shapes and joint connections, plus motor control parameters. A population of genomes is evolved over generations using a standard GA cycle.

1. Initialise a population of random genomes.
2. Decode each genome into a creature and simulate it in the mountain environment for a fixed horizon.
3. Compute fitness and log metrics for each individual.
4. Select parents biased toward higher fitness.
5. Apply crossover and mutation to produce offspring.
6. Replace the population, optionally keeping an elite individual.
7. Repeat until a fixed number of generations is reached.

In my implementation, experiments were run for 100 generations and each creature was evaluated for 2400 simulation steps per generation by default.

Reproducibility was handled by setting a random seed per run and saving a params.json file alongside a generation level log.

3. Fitness function and metrics

3.1 Primary metric: best supported distance to peak

The main performance measure is the best distance achieved to the mountain peak while the creature is supported by the terrain. At each timestep, the Euclidean distance from the creature's base position to the peak location is computed. The minimum distance observed across the episode gives best_dist. To reduce reward for unrealistic flight, I also compute best_supported_dist, which only updates the minimum distance when the creature is supported by contact with the environment. This aligns with the required objective of getting as high as possible without flying.

In the report and tables, lower supported distance is better because it means the creature reached closer to the peak.

3.2 Anti cheating metric: air ratio

To make "no flying" measurable, I track airtime across the episode. For each timestep, the creature is marked as "in air" if it has no contact support. The air ratio is the fraction of steps that are in air.

$$\text{air ratio} = \frac{\text{air steps}}{\text{total steps}}$$

A low air ratio suggests that the creature's progress is mainly ground supported, which matches the task intent.

3.3 Logged curves

For each run, I log per generation values that support analysis of learning dynamics.

- Best fitness in the population, which shows the best discovered solution
- Mean fitness, which indicates convergence and population wide improvement
- Best supported distance, which is the primary task outcome
- Best air ratio, which checks whether success relies on airtime

These are the values visualised in the figures inserted in Sections 4 and 5.

4. Basic experiments on GA settings

This section satisfies the coursework requirement to test different GA and genome settings and to summarise their impact using graphs and tables.

4.1 Experimental design

I performed a population size sweep with pop sizes 10, 25 and 50. Each configuration was run with three seeds to measure stability. Across these runs, the encoding scheme was held constant as mixed shapes with per joint control and full evolvability. Each run used 100 generations and 2400 iterations per creature evaluation.

For fair comparison, Table 1 reports mean and standard deviation across the three seeds for each population size. Figures show seed 1 learning curves for the same configurations to make the relative progression easier to see.

4.2 Results

pop_size	n	mean_final_best_supported_dist	std_final_best_supported_dist	mean_final_best_air_ratio	std_final_best_air_ratio
10	3	0.04267	0.03517	0.088	0.007
25	3	0.00247	0.00074	0.098	0.016
50	3	0.00179	0.00144	0.094	0.014

Table 1: Population size sweep summary across three seeds.
Columns include mean and standard deviation of final best supported distance and final best air ratio.

From Table 1, population size has a strong effect on the final supported distance. Pop 10 performs substantially worse than the larger populations, with mean final supported distance of 0.04267 and high variability. Pop 25 and pop 50 both achieve very small supported distances, with pop 25 at 0.00247 and pop 50 at 0.00179 on average. Pop 50 is slightly better in mean performance, but pop 25 is slightly more stable since its standard deviation is lower than pop 50. In practice, both pop 25 and pop 50 reach near peak solutions, while pop 10 tends to get stuck in weaker behaviours.

Airtime is similar across population sizes. The mean final air ratio is around 0.09 to 0.10 for all three settings, suggesting that improved performance in the larger populations is not achieved by increased airtime.

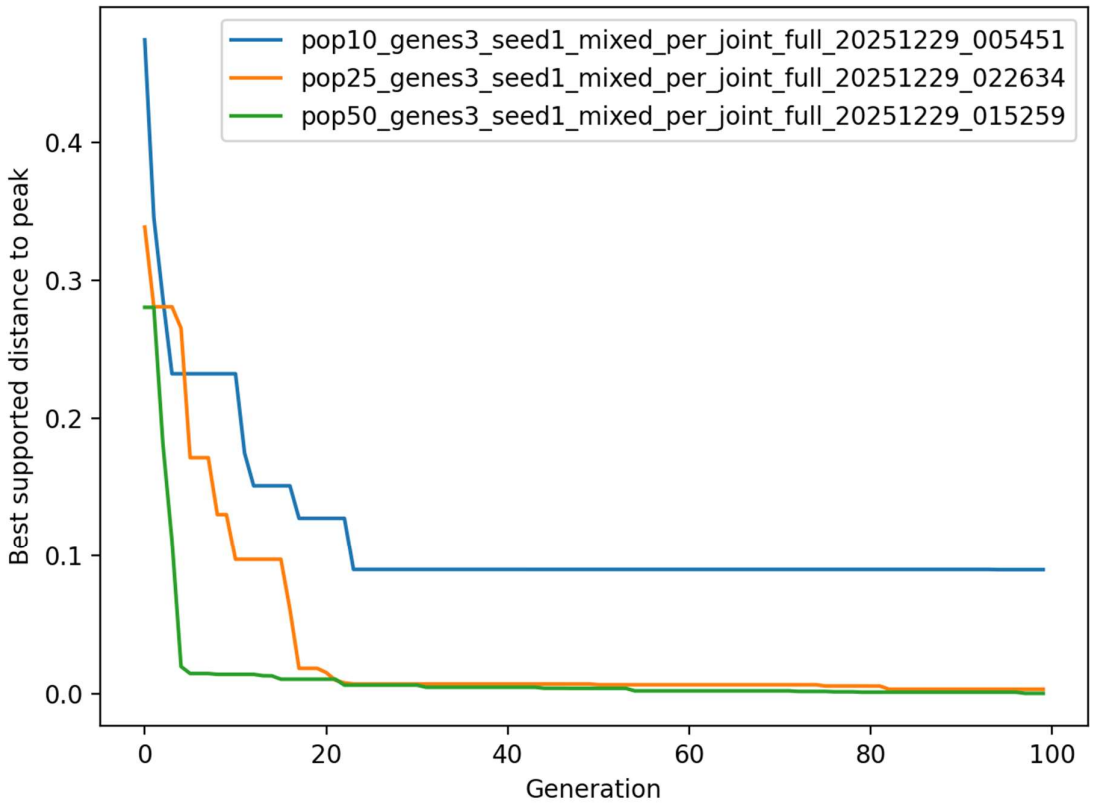

Figure 1: Best supported distance vs generation for population sizes 10, 25 and 50, seed 1.

Figure 1 shows that pop 25 and pop 50 reduce supported distance quickly in early generations and then plateau near very small values, consistent with convergence to effective climbing solutions. Pop 10 improves more slowly and plateaus at a much larger distance, which matches the weaker mean results in Table 1.

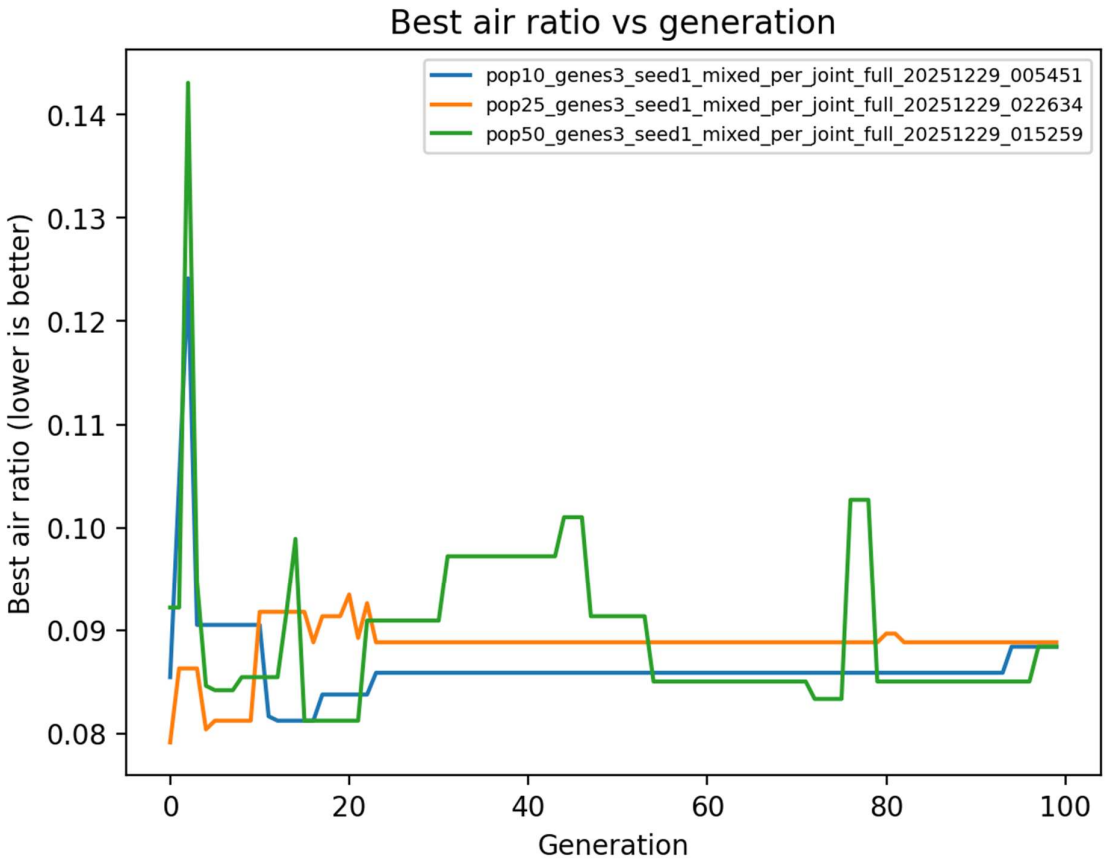

Figure 2: Best air ratio vs generation for population sizes 10, 25 and 50, seed 1.

Figure 2 provides evidence that the best individuals remain mostly ground supported. Air ratio values stay low across the run, and there is no clear trend where performance improvements coincide with large increases in airtime. This supports the claim that climbing progress reflects terrain supported locomotion rather than a flying exploit.

The best fitness and mean fitness curves show that pop 25 and pop 50 have stronger optimisation pressure and broader population improvement, which is consistent with larger population diversity.

5. Encoding scheme experiments

This section addresses the advanced coursework step of experimenting with the decoding and encoding scheme, including motor controls, shape of parts and which parts are evolvable.

5.1 Motivation

In a GA creatures system, the encoding defines what behaviours and morphologies can be expressed. Two systems with the same GA operators can produce very different outcomes if the genotype to phenotype mapping changes the search space. For mountain climbing, an encoding that allows useful leg shapes and effective control patterns should help reach the peak with stable contact.

5.2 Encodings tested

All encoding experiments used population size 25, gene count 3, 100 generations and three seeds for each condition. The conditions were:

- Mixed shapes, per joint control, full evolvability (baseline)
- Mixed shapes, global control, full evolvability
- Cylinder only shapes, per joint control, full evolvability
- Mixed shapes, per joint control, legs only evolvable
- Mixed shapes, per joint control, control only evolvable

These choices match the brief's suggested directions of varying motor controls, part shapes and evolvable subsets.

5.3 Results

shape_mode	control_scheme	encoding	n	mean_final_best_supported_dist	std_final_best_supported_dist	mean_final_best_air_ratio	std_final_best_air_ratio
cylinder_only	per_joint	full	3	0.01811	0.02218	0.084	0.006
mixed	global	full	3	0.00439	0.00265	0.107	0.009
mixed	per_joint	control_only	3	0.08160	0.03790	0.097	0.011
mixed	per_joint	full	3	0.00247	0.00074	0.098	0.016
mixed	per_joint	legs_only	3	0.00373	0.00159	0.091	0.013

Table 2: Encoding scheme experiment summary across three seeds.
Columns include mean and standard deviation of final best supported distance and final best air ratio.

Table 2 shows that encoding choices strongly affect mountain climbing success. The baseline encoding achieves the best overall supported distance on average at 0.00247. The global control scheme is slightly worse at 0.00439 and it has a higher mean air ratio of 0.1066, which suggests that coupling all joints to a shared control signal can reduce fine control and can also increase unstable motion.

The cylinder only shape restriction performs notably worse with mean supported distance 0.0181. However, it achieves the lowest air ratio at 0.0845, which suggests that restricting shapes can limit risky dynamics but also removes useful morphology options for climbing. The control only encoding performs worst by a large margin at 0.0816, which indicates that evolving motor parameters alone without morphology evolution is not sufficient for this task under these settings. The legs only condition performs close to the baseline at 0.00373 and has a slightly lower air ratio than baseline at 0.0910, which suggests that keeping the body fixed while evolving legs can preserve performance while improving stability.

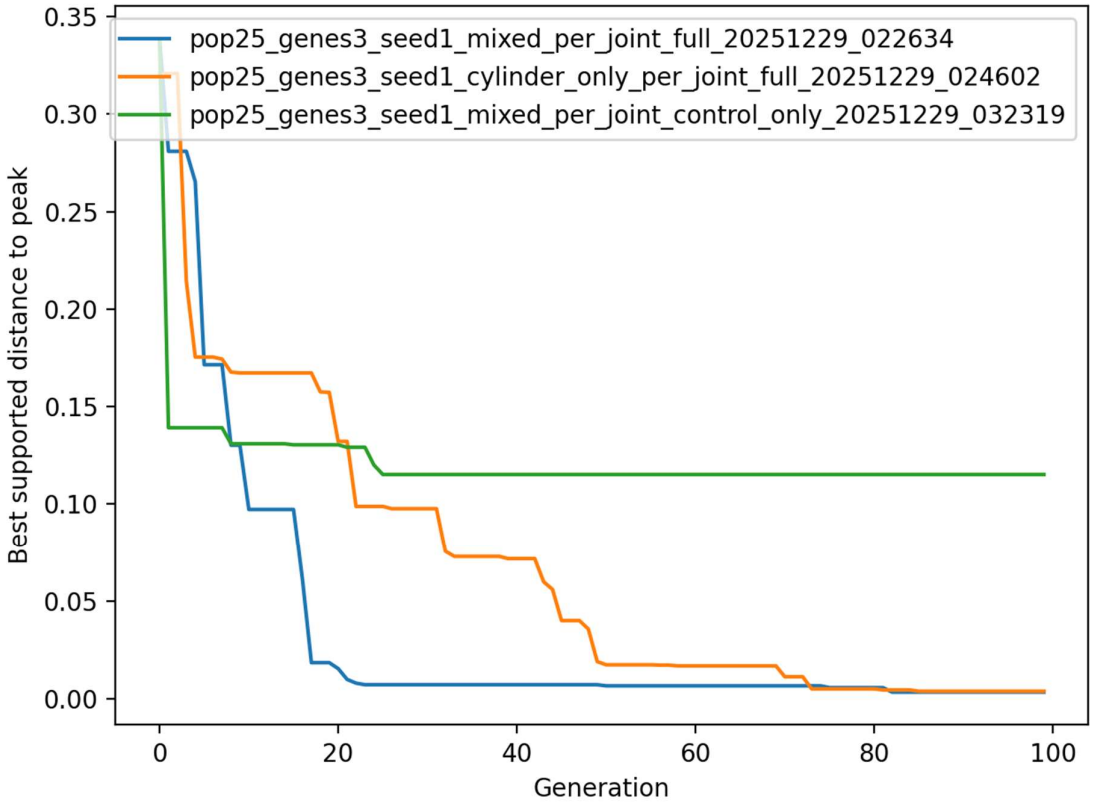

Figure 3: Best supported distance vs generation for key encoding conditions, seed 1.

Figure 3 illustrates how learning dynamics change with encoding. The baseline condition reduces supported distance rapidly and stabilises near a very small value. The cylinder only and control only conditions show slower improvement and worse plateaus, which aligns with their poorer final values in Table 2. The global control condition often improves early but reaches a higher plateau than baseline, which is consistent with reduced control resolution.

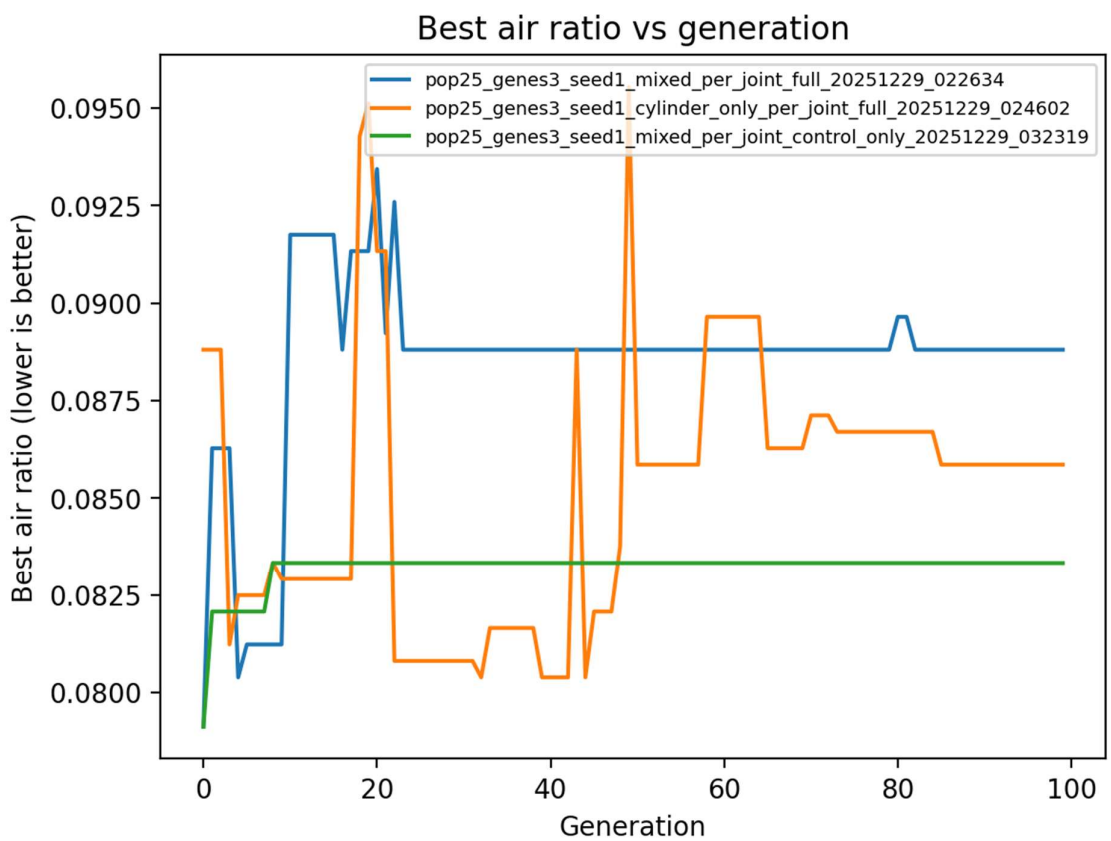

Figure 4: Best air ratio vs generation for key encoding conditions, seed 1.

Figure 4 helps interpret whether performance depends on airtime. Cylinder only remains low in air ratio but does not climb as well. Global control tends to have higher air ratio than baseline, which suggests more jumping or unstable motion during optimisation. Baseline and legs only maintain relatively low air ratio while still reaching near peak supported distance, which is the strongest evidence of valid climbing behaviour.

6. Discussion and limitations

Across experiments, larger populations improve climbing performance because they increase diversity and reduce the chance of premature convergence. This is clear from the large gap between pop 10 and the two larger settings in Table 1, and from the faster reductions in supported distance seen in Figure 1. The difference between pop 25 and pop 50 is smaller. Pop 50 gives slightly better mean supported distance, while pop 25 shows slightly lower variance, so pop 25 is a reasonable default when runtime is constrained.

Encoding experiments show that morphology evolution is critical. The control only setting performs poorly, which implies that locomotion solutions require structural adaptation rather than control tuning alone. Shape restrictions also reduce performance, which suggests that the richer mixed shape space contains useful configurations for climbing. The legs only result is important because it suggests that partial evolvability can retain most of the performance while reducing the search space and improving stability, which can be useful for computational efficiency.

There are limitations. Results are based on three seeds per condition, so estimates of stability are limited. The supported distance metric depends on how support is defined from contacts, so alternative definitions might change conclusions slightly. Furthermore, this implementation that I have done shows only very small improvements in distance made towards the mountain, needing hundreds if not thousands of generations to show any real progress, which is not quite feasible in this project due to the massive file size recording that data would create, not to mention the time taken to conduct the experiments on a normal computer. Finally, the evaluation horizon and physics settings can bias the GA toward short horizon behaviours, which may not generalise across different terrain shapes.

7. Conclusion

This project successfully adapted the GA creatures system to the mountain environment and produced creatures that can climb toward the peak without relying on high airtime.

Population size strongly affects performance, with pop 25 and pop 50 reaching near peak supported distances while pop 10 performs substantially worse. Encoding choices have an equally large impact. The baseline mixed per joint encoding is best overall, while restricting morphology or evolving control alone reduces performance. The legs only encoding remains competitive, suggesting that partial evolvability can be a strong compromise for future work.

8. References

Coumans, E. and Bai, Y. (n.d.) PyBullet: Python module for Bullet Physics simulation. Bullet Physics / PyBullet project documentation.

Harris, C.R., Millman, K.J., van der Walt, S.J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N.J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M.H., Brett, M., Haldane, A., del Río, J.F., Wiebe, M., Peterson, P., Gérard-Marchant, P., Sheppard, K., Reddy, T., Weckesser, W., Abbasi, H., Gohlke, C. and Oliphant, T.E. (2020) 'Array programming with NumPy', Nature, 585, pp. 357–362.

Hunter, J.D. (2007) 'Matplotlib: A 2D graphics environment', Computing in Science & Engineering, 9(3), pp. 90–95.